



Quay lui & Duyệt toàn bộ (Complete Search)

Nội dung

- 1. Ý tưởng: Giống như tìm đường trong mê cung
- 2. Template tổng quát của Backtracking
- 3. Sinh tất cả xâu nhị phân độ dài $\backslash(n\backslash)$
 - 3.1. Bài toán
 - 3.2. Ý tưởng
 - 3.3. Cây đệ quy cho $\backslash(n = 3\backslash)$
 - 3.4. Code
- 4. Sinh hoán vị (Permutations)
 - 4.1. Bài toán
 - 4.2. Ý tưởng
 - 4.3. Cây đệ quy cho $\backslash(n = 3\backslash)$
 - 4.4. Code
 - 4.5. Cách khác: next_permutation
- 5. Duyệt tập con bằng Bitmask
 - 5.1. Ý tưởng
 - 5.2. Ví dụ với $\backslash(n = 3\backslash)$, tập $\backslash(\{5, 3, 7\}\backslash)$
 - 5.3. Các phép bit cơ bản
 - 5.4. Code duyệt tất cả tập con
 - 5.5. Ví dụ: chia tập thành 2 nhóm có chênh lệch nhỏ nhất
- 6. Ví dụ kinh điển: N-Queens (Xếp Hậu)
 - 6.1. Bài toán
 - 6.2. Phân tích
 - 6.3. Code
 - 6.4. Giải thích đường chéo
 - 6.5. Số lời giải một vài giá trị nhỏ
- 7. Pruning — Cắt tỉa để tăng tốc
 - 7.1. Pruning là gì?



- 7.3. Các kỹ thuật pruning phổ biến

- 8. Sinh tổ hợp $\binom{n}{k}$
 - 8.1. Ý tưởng
 - 8.2. Code
 - 8.3. Pruning cho tổ hợp
- 9. Các lỗi phổ biến
- Tóm tắt bài học

Trong bài học này, chúng ta sẽ tìm hiểu kỹ thuật **Backtracking** (quay lui) — một kỹ thuật duyệt có hệ thống tất cả các khả năng. Khi giới hạn n đủ nhỏ, thay vì tìm thuật toán thông minh, ta có thể **thử tất cả** và chọn đáp án tốt nhất!

1. Ý tưởng: Giống như tìm đường trong mê cung

Hãy tưởng tượng bạn đang đứng trong một mê cung và cần tìm đường ra. Bạn sẽ làm gì?

- Bước 1: Đi theo một hướng (ví dụ: rẽ trái)

Bước 2: Nếu đi tiếp được → tiếp tục đi

Bước 3: Nếu gặp ngõ cụt → QUAY LUI về ngã rẽ trước đó

Bước 4: Thử hướng khác (ví dụ: rẽ phải)

Bước 5: Lặp lại cho đến khi tìm thấy lối ra (hoặc đã thử hết)

Đó chính là **backtracking**: thử một lựa chọn → đi sâu → nếu thất bại → **quay lui** (undo) → thử lựa chọn khác.

Khi nào dùng backtracking?

- Khi n nhỏ
- Khi cần tìm **tất cả** lời giải
- Khi không có thuật toán hiệu quả hơn
- Khi bài toán có dạng "thử mọi khả năng nhưng có điều kiện"

Bảng tham khảo:



Tiếng Việt



Xâu nhị phân	2^n	$n \leq 20$
Tập con	2^n	$n \leq 20$
Hoán vị	$n!$	$n \leq 10$
Tổ hợp $C(n, k)$	$\binom{n}{k}$	Tùy n, k

2. Template tổng quát của Backtracking

Hầu hết các bài backtracking đều theo cấu trúc sau:

C++

Python



```
int result[MAXN];    // Lời giải đang xây dựng
bool used[MAXN];     // Đánh dấu đã dùng chưa (nếu cần)

void backtrack(int pos) {
    // Bước 1: Kiểm tra đã hoàn thành chưa
    if (pos == n) {
        // Xử lý lời giải (in, cập nhật đáp án, ...)
        return;
    }

    // Bước 2: Thử tất cả lựa chọn cho vị trí pos
    for (/* mỗi lựa chọn c */) {
        if (/* c hợp lệ */) {
            // Bước 3: Chọn
            result[pos] = c;

            // Bước 4: Đi tiếp
            backtrack(pos + 1);

            // Bước 5: QUAY LUI
        }
    }
}
```



```
used = [False] * (n + 1)

def backtrack(pos):
    if pos == n:
        # xử lý lời giải
        return

    for c in choices:
        if valid(c):
            result[pos] = c
            backtrack(pos + 1)
            # quay lui nếu cần
```

Điểm mấu chốt: bước **quay lui** là bắt buộc nếu bạn có thay đổi trạng thái toàn cục như `used`, `sum`, `path`, ...

3. Sinh tất cả xâu nhị phân độ dài n

3.1. Bài toán

In tất cả xâu gồm n ký tự, mỗi ký tự là `'0'` hoặc `'1'`.

Có đúng 2^n xâu.

3.2. Ý tưởng

Tại mỗi vị trí, ta có 2 lựa chọn:

- đặt `'0'`
- đặt `'1'`

Thử cả hai lựa chọn bằng đệ quy.

3.3. Cây đệ quy cho $n = 3$



Tiếng Việt



```

      /      \
    "0"      "1"
    /  \    /  \
  "00" "01" "10" "11"
  /  \  /  \  /  \  /  \
000 001 010 011 100 101 110 111

```

3.4. Code

C++

Python

```

#include <bits/stdc++.h>
using namespace std;

int n;
string s;

void generate(int pos) {
    if (pos == n) {
        cout << s << "\n";
        return;
    }

    s[pos] = '0';
    generate(pos + 1);

    s[pos] = '1';
    generate(pos + 1);
}

int main() {
    cin >> n;
    s.resize(n);
    generate(0);
}

```



Tiếng Việt



```
s = [''] * n

def generate(pos):
    if pos == n:
        print(''.join(s))
        return

    s[pos] = '0'
    generate(pos + 1)

    s[pos] = '1'
    generate(pos + 1)

generate(0)
```

Input:

Output:

```
000
001
010
011
100
101
110
111
```

4. Sinh hoán vị (Permutations)

4.1. Bài toán

In tất cả hoán vị của $\{1, 2, \dots, n\}$.

Có $n!$ hoán vị.



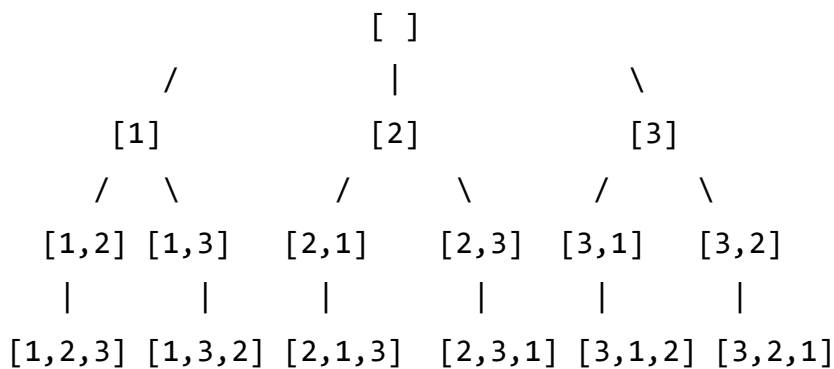
Tiếng Việt



Ở vị trí `pos`, thử đặt từng số từ `1` đến `n` mà **chưa dùng**.

Sau khi đệ quy xong, nhớ **bỏ đánh dấu** để thử số khác.

4.3. Cây đệ quy cho $n = 3$



4.4. Code

C++

Python



```
using namespace std;

const int MAXN = 15;
int n;
int perm[MAXN];
bool used[MAXN];

void generate(int pos) {
    if (pos == n) {
        for (int i = 0; i < n; i++) {
            cout << perm[i] << " ";
        }
        cout << "\n";
        return;
    }

    for (int i = 1; i <= n; i++) {
        if (!used[i]) {
            used[i] = true;
            perm[pos] = i;

            generate(pos + 1);

            used[i] = false;    // quay lui
        }
    }
}

int main() {
    cin >> n;
    generate(0);
}
```



Tiếng Việt



```
perm = [0] * n
used = [False] * (n + 1)

def generate(pos):
    if pos == n:
        print(*perm)
        return

    for i in range(1, n + 1):
        if not used[i]:
            used[i] = True
            perm[pos] = i

            generate(pos + 1)

            used[i] = False    # quay lui

generate(0)
```

4.5. Cách khác: `next_permutation`

Nếu chỉ cần duyệt tất cả hoán vị theo thứ tự từ điển, C++ có thể dùng `next_permutation`.

C++



```
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    iota(a.begin(), a.end(), 1); // a = {1, 2, ..., n}

    do {
        for (int i = 0; i < n; i++) {
            cout << a[i] << " ";
        }
        cout << "\n";
    } while (next_permutation(a.begin(), a.end()));
}
```

5. Duyệt tập con bằng Bitmask

5.1. Ý tưởng

Mỗi tập con của n phần tử có thể biểu diễn bằng một số nguyên gồm n bit.

- Bit thứ i bằng 1: chọn phần tử a_i
- Bit thứ i bằng 0: không chọn

Vì có 2 lựa chọn cho mỗi phần tử, tổng cộng có 2^n tập con.

5.2. Ví dụ với $n = 3$, tập $\{5, 3, 7\}$

MASK (NHỊ PHÂN)	MASK (THẬP PHÂN)	TẬP CON	TỔNG
000	0	{}	0
001	1	{5}	5
010	2	{3}	3
011	3	{5, 3}	8



Tiếng Việt



100	4	{7}	7
101	5	{5, 7}	12
110	6	{3, 7}	10
111	7	{5, 3, 7}	15

5.3. Các phép bit cơ bản

PHÉP TOÁN	Ý NGHĨA
<code>mask >> i & 1</code>	Kiểm tra bit thứ i
<code>mask (1 << i)</code>	Bật bit thứ i
<code>mask & ~(1 << i)</code>	Tắt bit thứ i
<code>mask ^ (1 << i)</code>	Đảo bit thứ i
<code>1 << n</code>	Bằng 2^n
<code>__builtin_popcount(mask)</code>	Đếm số bit 1

5.4. Code duyệt tất cả tập con

C++

Python



Tiếng Việt



```
int a[25];

for (int mask = 0; mask < (1 << n); mask++) {
    int sum = 0;
    vector<int> subset;

    for (int i = 0; i < n; i++) {
        if (mask >> i & 1) {
            sum += a[i];
            subset.push_back(a[i]);
        }
    }

    // xử lý subset hoặc sum
}
```

```
for mask in range(1 << n):
    subset = []
    total = 0
    for i in range(n):
        if (mask >> i) & 1:
            subset.append(a[i])
            total += a[i]
    # xử lý subset hoặc total
```

5.5. Ví dụ: chia tập thành 2 nhóm có chênh lệch nhỏ nhất

C++

Python



```

for (int i = 0; i < n; i++) total += a[i];

int min_diff = total;

for (int mask = 0; mask < (1 << n); mask++) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        if (mask >> i & 1) sum += a[i];
    }
    min_diff = min(min_diff, abs(total - 2 * sum));
}

cout << min_diff << "\n";

```

```

total = sum(a)
min_diff = total

for mask in range(1 << n):
    s = 0
    for i in range(n):
        if (mask >> i) & 1:
            s += a[i]
    min_diff = min(min_diff, abs(total - 2 * s))

print(min_diff)

```

6. Ví dụ kinh điển: N-Queens (Xếp Hậu)

6.1. Bài toán

Đặt N quân Hậu lên bàn cờ $N \times N$ sao cho không quân nào tấn công quân nào.

Một quân Hậu tấn công theo:

- cùng hàng
- cùng cột



Tiếng Việt



6.2. Phân tích

Vì mỗi hàng chỉ cần đặt 1 Hậu, ta sẽ duyệt từng hàng một.

Ở hàng `row`, thử đặt Hậu vào từng cột `c` nếu:

- cột `c` chưa có Hậu
- đường chéo chính chưa bị chiếm
- đường chéo phụ chưa bị chiếm

6.3. Code

C++

Python



```
using namespace std;

int n, count_solutions = 0;
int col[20];
bool used_col[20];
bool used_diag1[40];
bool used_diag2[40];

void solve(int row) {
    if (row == n) {
        count_solutions++;
        return;
    }

    for (int c = 0; c < n; c++) {
        if (!used_col[c] && !used_diag1[row - c + n] && !used_diag2[row + c]) {
            col[row] = c;
            used_col[c] = true;
            used_diag1[row - c + n] = true;
            used_diag2[row + c] = true;

            solve(row + 1);

            used_col[c] = false;
            used_diag1[row - c + n] = false;
            used_diag2[row + c] = false;
        }
    }
}

int main() {
    cin >> n;
    solve(0);
    cout << count_solutions << "\n";
}
```




```
count_solutions = 0

used_col = [False] * n
used_diag1 = [False] * (2 * n)
used_diag2 = [False] * (2 * n)

def solve(row):
    global count_solutions

    if row == n:
        count_solutions += 1
        return

    for c in range(n):
        d1 = row - c + n
        d2 = row + c

        if not used_col[c] and not used_diag1[d1] and not used_diag2[d2]:
            used_col[c] = True
            used_diag1[d1] = True
            used_diag2[d2] = True

            solve(row + 1)

            used_col[c] = False
            used_diag1[d1] = False
            used_diag2[d2] = False

solve(0)
print(count_solutions)
```

6.4. Giải thích đường chéo

- Đường chéo chính: các ô có cùng giá trị $row - col$
- Đường chéo phụ: các ô có cùng giá trị $row + col$

Vì $row - col$ có thể âm, ta cộng thêm n để tránh chỉ số âm.



Tiếng Việt



N	SỐ LỜI GIẢI
1	1
2	0
3	0
4	2
5	10
6	4
7	40
8	92

7. Pruning — Cắt tỉa để tăng tốc

7.1. Pruning là gì?

Pruning là kỹ thuật bỏ qua những nhánh mà ta biết chắc sẽ không tạo ra đáp án tốt hoặc không thể tạo ra lời giải hợp lệ.

Ý tưởng là: **không cần thử hết nếu biết trước nhánh đó vô ích.**

7.2. Ví dụ: tìm tập con có tổng bằng S

Không pruning:

C++

Python



```
if (pos == n) {
    if (current_sum == S) count++;
    return;
}

backtrack(pos + 1, current_sum + a[pos]);
backtrack(pos + 1, current_sum);
}
```

```
def backtrack(pos, current_sum):
    global count
    if pos == n:
        if current_sum == S:
            count += 1
        return

    backtrack(pos + 1, current_sum + a[pos])
    backtrack(pos + 1, current_sum)
```

Có pruning:

C++

Python

```
void backtrack(int pos, int current_sum) {
    if (current_sum > S) return;
    if (current_sum + remaining_sum[pos] < S) return;

    if (pos == n) {
        if (current_sum == S) count++;
        return;
    }

    backtrack(pos + 1, current_sum + a[pos]);
    backtrack(pos + 1, current_sum);
}
```



Tiếng Việt



```
global count

if current_sum > S:
    return

if current_sum + remaining_sum[pos] < S:
    return

if pos == n:
    if current_sum == S:
        count += 1
    return

backtrack(pos + 1, current_sum + a[pos])
backtrack(pos + 1, current_sum)
```

7.3. Các kỹ thuật pruning phổ biến

1. **Vượt giới hạn:** ví dụ tổng đã lớn hơn S
2. **Không đủ khả năng đạt đáp án:** kể cả lấy hết phần còn lại vẫn không đủ
3. **Sắp xếp trước:** để tìm lời giải tốt sớm hơn
4. **Loại bỏ đối xứng:** tránh duyệt hai nhánh tương đương nhau

8. Sinh tổ hợp $C(n, k)$

Bài toán: chọn k phần tử từ $\{1, 2, \dots, n\}$.

8.1. Ý tưởng

Ta chọn phần tử đầu tiên, rồi chọn phần tử tiếp theo lớn hơn nó để tránh trùng.

8.2. Code

C++

Python



```
using namespace std;

int n, k;
int combo[25];

void generate(int pos, int start) {
    if (pos == k) {
        for (int i = 0; i < k; i++) {
            cout << combo[i] << " ";
        }
        cout << "\n";
        return;
    }

    for (int i = start; i <= n; i++) {
        combo[pos] = i;
        generate(pos + 1, i + 1);
    }
}

int main() {
    cin >> n >> k;
    generate(0, 1);
}
```



Tiếng Việt



```
combo = [0] * k

def generate(pos, start):
    if pos == k:
        print(*combo)
        return

    for i in range(start, n + 1):
        combo[pos] = i
        generate(pos + 1, i + 1)

generate(0, 1)
```

Ví dụ: $C(4, 2)$

```
1 2
1 3
1 4
2 3
2 4
3 4
```

8.3. Pruning cho tổ hợp

C++

Python



Tiếng Việt



```

if (n - start + 1 < k - pos) return;

if (pos == k) {
    for (int i = 0; i < k; i++) {
        cout << combo[i] << " ";
    }
    cout << "\n";
    return;
}

for (int i = start; i <= n; i++) {
    combo[pos] = i;
    generate(pos + 1, i + 1);
}
}

```

```

def generate(pos, start):
    if n - start + 1 < k - pos:
        return

    if pos == k:
        print(*combo)
        return

    for i in range(start, n + 1):
        combo[pos] = i
        generate(pos + 1, i + 1)

```

9. Các lỗi phổ biến

LỖI	HẬU QUẢ	CÁCH TRÁNH
Quên quay lui	Kết quả sai hoặc thiếu lời giải	Luôn hoàn tác sau đệ quy



Tiếng Việt



n quá lớn cho brute force	TLE	Ước lượng $2^n, n!$ trước
Sinh trùng lặp	Đếm thừa, output sai	Dùng <code>used</code> hoặc <code>start</code>
Không xử lý edge case	WA / RE	Kiểm tra $n = 0, k = 0, \dots$
Đệ quy quá sâu	Stack overflow	Cẩn thận khi n lớn

Tóm tắt bài học

KỸ THUẬT	SỐ TRẠNG THÁI	GIỚI HẠN AN TOÀN	KHI NÀO DỪNG
Xâu nhị phân / Tập con	2^n	$n \leq 20$	Chọn / không chọn
Hoán vị	$n!$	$n \leq 10$	Sắp xếp thứ tự
Bitmask	2^n	$n \leq 20$	Duyệt tập con
Tổ hợp $C(n, k)$	$\binom{n}{k}$	Tùy bài	Chọn k phần tử
N-Queens	Phụ thuộc pruning	$n \leq 15$	Bài toán ràng buộc
Pruning	Giảm đáng kể	-	Tăng tốc duyệt

Bây giờ hãy thử sức với các bài tập! Bắt đầu từ sinh xâu nhị phân và hoán vị, rồi tiến đến các bài cần kết hợp backtracking với pruning nhé. Chúc bạn AC!

Bài tập

Sinh nhị phân	0 / 10
Sinh hoán vị	0 / 10
Tổng dãy con bằng K	0 / 10
CSES - Creating Strings Tạo xâu	0 / 15
CSES - Tower of Hanoi Tháp Hà Nội	0 / 15



Tiếng Việt



 Có phải số Fibo?	0 / 15
 CSES - Chessboard and Queens Bàn cờ và quân hậu	0 / 20
 CSES - Apple Division Chia táo	0 / 20
 Dây ngoặc	0 / 20
 Chia Bò Sữa	0 / 25
 Biểu thức 1	0 / 25

proudly powered by **DMOJ** | [developed by LQDJudge team](#) Tiếng Việt (vi) ▼